

Chart Libraries for Enterprise HRMS Dashboards

Executive Summary

For an enterprise HRMS dashboard, we recommend **Recharts** (React/D3 SVG), **Apache ECharts** (Apache-licensed, Canvas/SVG), and (optionally) the new **TanStack Charts** (TypeScript-first). Recharts has broad adoption and a simple React API, making it a “safe default” for general-purpose dashboards [24†L769-L775] [21†L255-L263] . Apache ECharts excels on large real-time datasets and complex chart types (with zoom, pan, and progressive rendering) [22†L539-L547] [22†L557-L566] . TanStack Charts is a modern, TS-native grammar-of-graphics library (MIT-licensed) promising responsive, accessible charts by default [6†L496-L499] [13†L222-L231] .

As fallbacks, **react-chartjs-2 (Chart.js)** and **visx** are worth considering. Chart.js (via react-chartjs-2) is Canvas-based and lightweight (if tree-shaken) [24†L745-L749] , great for high-frequency updates on SPAs [21†L329-L334] . Visx (Airbnb) provides low-level SVG primitives with excellent bundle control [24†L743-L748] [24†L655-L663] , ideal if you need fully custom visualizations integrated with Tailwind.

All libraries have tradeoffs. SVG-based libs (Recharts, TanStack, Visx) generally support SSR and integrate well with React UI stacks [24†L769-L775] [13†L222-L231] , but can struggle with very large datasets (Canvas-based ECharts/Chart.js perform better there). None of these libraries auto-solve accessibility – for example, Recharts v3 offers a built-in `accessibilityLayer` [21†L261-L264] but Ashlee Boyer notes screen-reader support is still incomplete [17†L139-L148] . In practice you should follow WCAG guidelines (ARIA labels, keyboard focus, color contrast, alt text) alongside the library’s support.

Below is a summary comparison table and detailed evaluation of each contender, including example usage and theming tips. We prioritize libraries with strong TypeScript support, React-first APIs, and good integration with Tailwind/shadcn-based design systems.

Comparison Table

Library	Rendering	Bundle (gz)	License	React API	Notable Use Cases
Recharts	SVG	~50 KB (moderate) [24†L753-L757]	MIT (OSS)	Native React components [21†L249-L253]	General React dashboards; shadcn-ui projects [24†L769-L775]
Apache ECharts	SVG/Canvas	Large (80+KB naive import) [24†L757-L760]	Apache 2.0 (OSS)	Vanilla JS (use wrapper)	Large/real-time data, interactive analytics [22†L539-L547] [22†L557-L566]

Library	Rendering	Bundle (gz)	License	React API	Notable Use Cases
TanStack Charts	SVG (default), Canvas optional 【13†L222-L231】	Small (modular grammar)	MIT (OSS)	Native TS+React chart definitions 【6†L496-L499】 【13†L208-L216】	High customization; accessible, SSR-friendly charts
react-chartjs-2	Canvas	~40 KB (treeshaken) 【24†L745-L749】	MIT (OSS)	React wrapper for Chart.js 【21†L329-L334】	Real-time/SPAs (many points) 【21†L329-L334】
visx	SVG	Smallest (primitives) 【24†L743-L748】	MIT (OSS)	Low-level SVG primitives	Custom design systems; bundle-critical charts 【24†L762-L765】

- **Rendering Modes:** Recharts/TanStack/Visx render via SVG (good for style, SSR); ECharts and Chart.js primarily use Canvas (better for high-volume data) 【21†L255-L259】 【22†L539-L547】 .
- **Bundle Size:** visx (if you import only what you need) is usually smallest 【24†L743-L748】 . Recharts is convenient but not very tree-shakeable 【24†L753-L757】 . ECharts/Ant Charts are heavy unless you carefully import individual components 【24†L757-L760】 . Chart.js can be moderate if you only register needed chart types 【24†L745-L749】 .
- **React/TypeScript:** Recharts, TanStack, Chart.js, Visx all have TypeScript support and React bindings. Recharts and TanStack offer a declarative React API 【21†L249-L253】 【13†L208-L216】 . ECharts has no official React component, but third-party wrappers (e.g. echarts-for-react) exist.
- **Accessibility:** Only AG Charts (not shown) advertises built-in ARIA support 【20†L159-L164】 . Recharts v3 improved its accessibilityLayer 【21†L261-L264】 , but studies find screen-reader output is still incomplete 【17†L139-L148】 【10†L139-L147】 . TanStack requires an `ariaLabel` 【13†L268-L277】 for the chart surface. Overall, none fully replace careful a11y design (color contrast, labels, captions).
- **Use Cases:** Recharts is the “safest default” for typical dashboards 【24†L769-L775】 . ECharts (and Chart.js) shine with very large or streaming data and advanced interactions (zoom, pan, real-time) 【22†L539-L547】 【21†L329-L334】 . TanStack/visx best for custom design-system integration and minimal overhead.

With these in mind, our top picks (Recharts, ECharts, TanStack) offer a strong balance of React compatibility, chart variety, theming potential, and TypeScript support. Chart.js (react-chartjs-2) and visx are excellent fallback options for specialized scenarios (real-time heavy data or highly-custom UIs).

Recharts (React + D3, SVG)

Why: Recharts is a mature, React-centric library with broad adoption. Its API is idiomatic to React (compose `<LineChart>`, `<BarChart>`, etc.) 【21†L249-L257】 and it handles axes, legends, and tooltips out of the box. Shadcn UI (Tailwind-based) even provides reusable chart components built on Recharts 【16†L169-

L177】 . Recharts v3 now includes an “accessibilityLayer” enabled by default 【21†L261-L264】 , improving keyboard navigation.

Pros: Native React components; good defaults and customization; SVG rendering enables CSS styling, server-side rendering, and crisp vector output 【21†L255-L263】 . Mature documentation and community (27K★ on GitHub) help with support. Works well with Tailwind/shadcn UI (see Ashlee Boyer’s review of shadcn charts) 【16†L169-L177】 【17†L69-L77】 .

Cons: Bundle size is moderate and not fully tree-shakeable 【24†L753-L757】 . SVG can lag with tens of thousands of points (Canvas-based libs handle those faster) 【24†L769-L775】 . Default charts have limited advanced interactions (no built-in zoom/brush), though basic tooltips and animations are supported. Accessibility still requires manual labeling for full WCAG compliance 【17†L139-L148】 .

Example (React + TS) – A multi-series line chart and stacked bar chart:

```
import { LineChart, Line, BarChart, Bar, XAxis, YAxis, Tooltip, Legend,
ResponsiveContainer } from 'recharts';

const sampleData = [
  { month: 'Jan', revenue: 2400, cost: 1800 },
  { month: 'Feb', revenue: 3200, cost: 2000 },
  { month: 'Mar', revenue: 2800, cost: 2200 },
  // ...
];

export function KPILineChart() {
  return (
    <ResponsiveContainer width="100%" height={300}>
      <LineChart data={sampleData}>
        <XAxis dataKey="month" />
        <YAxis />
        <Tooltip />
        <Legend />
        { /* Multiple series */ }
        <Line type="monotone" dataKey="revenue" stroke="#2563EB" />
        <Line type="monotone" dataKey="cost" stroke="#DC2626" />
      </LineChart>
    </ResponsiveContainer>
  );
}

export function KPIRadarAndBar() {
  return (
    <ResponsiveContainer width="100%" height={300}>
      <BarChart data={sampleData}>
        <XAxis dataKey="month" />
```

```

    <YAxis />
    <Tooltip />
    <Legend />
    { /* Stacked bars using stackId */}
    <Bar dataKey="revenue" stackId="a" fill="#2563EB" />
    <Bar dataKey="cost" stackId="a" fill="#DC2626" />
  </BarChart>
</ResponsiveContainer>
);
}

```

Use similar `<LineChart>` or `<BarChart>` components for small sparklines (simply reduce height to ~50px). Recharts components accept `className`/ `fill` props, so you can integrate Tailwind classes or custom color tokens (e.g. use `className="stroke-blue-600"` instead of `stroke="#2563EB"`). For theming, define CSS variables or React context to inject the HRMS color palette (e.g. `--primary`, `--danger`) and use them in the `fill` / `stroke`.

Performance Tips: Only import needed chart types. Use `ResponsiveContainer` (as above) for automatic resizing. For large datasets, consider downsampling before passing to Recharts or switching to a Canvas library. Recharts also supports `<ReferenceLine>` and `<Customized>` elements for thresholds/ annotations.

Accessibility: Always supply `aria-label` on the containing div or use the new `accessibilityLayer`. For example:

```

<ResponsiveContainer>
  <BarChart
    data={data}
    aria-label="Monthly revenue vs cost"
    role="application"
  >  </BarChart>
</ResponsiveContainer>

```

Ensure color contrasts meet AA (the default blue `#2563EB` and red `#DC2626` have good contrast on white). Provide textual descriptions for chart purpose.

Testing: Recharts charts can be unit-tested by rendering with test IDs or snapshot tests of the SVG output. Visual diffs (e.g. with Storybook/Cypress) can catch rendering regressions. Since Recharts outputs SVG, you can also use tools like `@testing-library/jest-dom` to verify presence of `<path>` elements or axis labels.

Apache ECharts

Why: Apache ECharts is a powerful, enterprise-grade library (Apache 2.0) known for performance on large data and rich interactions [22†L539-L547]. It supports line/area/bar/pie/heatmap/sankey/etc. out of the

box. Its rendering can be SVG or Canvas; by default it uses Canvas for performance, with progressive loading for big series [22†L547-L556] . ECharts excels at dense dashboards, real-time streaming, and mobile-friendly interactions (pinch-to-zoom, etc.) [22†L547-L556] .

Pros: Extensive chart types (even geographic and tree maps), built-in features (data zoom, toolbox, exporting). Strong theming support (you can define a JSON theme to match your design palette). Very performant with thousands of points (use `dataZoom` for user zooming, and `benchmark` for real-time metrics). It integrates with React via wrappers like [echarts-for-react](#) or by creating a ref to an ECharts instance. Licensing is permissive (Apache 2.0).

Cons: Large bundle by default. You **must** use the modular import API to tree-shake. As LogRocket notes, avoid `import * as echarts from 'echarts'` (which bundles ~800KB); instead do:

```
import * as echarts from 'echarts/core';
import { LineChart, BarChart } from 'echarts/charts';
import { TooltipComponent, GridComponent, LegendComponent } from 'echarts/
components';
import { CanvasRenderer } from 'echarts/renderers';

echarts.use([LineChart, BarChart, TooltipComponent, GridComponent,
LegendComponent, CanvasRenderer]);
```

This selects only needed chart types and features [22†L557-L566] . ECharts has no native React API (it predates React); UI is managed imperatively, but wrappers can abstract it.

Example (React + TS) – A line chart using `echarts-for-react` wrapper (with Canvas renderer):

```
import React from 'react';
import ReactECharts from 'echarts-for-react';

const option = {
  title: { text: 'Employee Headcount' },
  tooltip: { trigger: 'axis' },
  xAxis: { type: 'category', data: ['Q1', 'Q2', 'Q3', 'Q4'] },
  yAxis: { type: 'value' },
  series: [
    {
      name: 'Full-Time',
      type: 'line',
      data: [120, 132, 101, 134],
      smooth: true,
      color: '#2563EB',
    },
    {
      name: 'Contractors',
```

```

    type: 'line',
    data: [80, 90, 95, 100],
    smooth: true,
    color: '#DC2626',
  },
],
};
export function HeadcountChart() {
  return (
    <ReactECharts
      option={option}
      style={{ height: 300, width: '100%' }}
      notMerge={true}
      lazyUpdate={true}
    />
  );
}

```

In the code above we explicitly set colors to match the palette. For Bar (stacked) charts or other types, just change `type: 'bar'` and add `stack: 'dept'`. Table-sparklines can be done by rendering a tiny ECharts instance in a table cell (e.g. 100×50 px).

Theming: You can define a theme JSON that sets default colors, fonts, and backgrounds. For example:

```

echarts.registerTheme('hrms', {
  backgroundColor: '#F8F9FB',
  textStyle: { color: '#111827', fontFamily: 'Inter, sans-serif' },
  color: ['#2563EB', '#4B5563', '#DC2626', '#D97706'],
  // ... configure axis lines, grid color (#E5E7EB) etc.
});

```

Then use `<ReactECharts theme="hrms" ... />`. This ensures charts use the corporate palette and typography.

Performance: ECharts can handle tens of thousands of points with its Canvas renderer and progressive mode. Use the `dataZoom` and `connect` features for interactive exploration. On mobile, ECharts automatically supports touch gestures. For streaming updates, call `setOption` with `notMerge: false` to update data incrementally.

Accessibility: ECharts's Canvas output has no built-in ARIA. You should provide an `<aside>` or `<p>` description. Use `aria-label` on the container div for screen readers. For simple charts, consider overlaying an invisible table or summary. (Unlike Recharts, ECharts gives you no ready accessibility layer.)

Testing: You can test the `option` object logic easily since it's just a JS object. For visual tests, render the `<ReactECharts>` and use snapshots or image diff tools. Because ECharts builds its own canvas, unit tests

generally verify that correct configuration is passed (e.g. checking `chartInstance.getOption()` in a JSDOM test).

TanStack Charts (New TS-First Grammar)

Why: TanStack Charts is a newly released TypeScript-first charting library (by the creators of React Query/Table) [6†L496-L499] . It implements a grammar-of-graphics approach, offering composable chart definitions. It's designed for responsiveness, accessibility, and server-side rendering [6†L496-L499] [13†L222-L231] . For a forward-looking project (2026+), it promises strong typing and modern architecture.

Pros: Very flexible. Default rendering is SVG with an optional Canvas adapter [13†L222-L231] . It has built-in support for data transforms (aggregations) and view compositions. By default, it enforces accessible labels (`ariaLabel` is required) [13†L268-L277] . The TanStack ecosystem means good TypeScript support and interop with other TanStack tools. The bundle size is reported very small (core grammar, and you import only needed marks/types).

Cons: It's brand-new and not as battle-tested in the wild. Fewer ready-made components or community examples compared to Recharts. The learning curve (grammar-of-graphics) is steeper. There's not yet a "TinyBundle" community size to cite.

Usage: You define a chart by calling `defineChart()` with marks and channels. A simple example (line chart with two series):

```
import { Chart } from '@tanstack/charts/react';

const data = [
  { month: 'Jan', value1: 2400, value2: 1200 },
  { month: 'Feb', value1: 3200, value2: 1600 },
  // ...
];

// Define the chart specification
const definition = defineChart({
  data,
  x: { field: 'month', type: 'band' },
  y: { field: 'value1', type: 'linear' },
  series: [
    { id: 'Full-Time', field: 'value1', color: '#2563EB' },
    { id: 'Part-Time', field: 'value2', color: '#DC2626' },
  ],
});

export function EmployeesLineChart() {
  return (
    <Chart
```

```

    definition={definition}
    ariaLabel="Monthly HR metrics"
    width={500}
    height={300}
  />
);
}

```

(This is a simplified illustration; the actual API uses `ChartDefinition` objects. The docs have more examples.) By using CSS variables or React theme, you can inject the HRMS color tokens into the definition (e.g. `color: 'var(--primary)'`).

Theming & Design System: TanStack's chart API accepts `className` and `style` on the container [【13†L272-L279】](#), so you can wrap it in a Tailwind-styled div. For legends or tooltips, you supply custom components (see the `/tooltip` entry in docs). This lets you reuse the HRMS design tokens in every chart.

Performance: Since it defaults to SVG, it's SSR-friendly and suits moderate data sizes. For extreme data, switch to `CanvasChart` from `@tanstack/charts/react/canvas` [【13†L227-L235】](#). TanStack Charts is designed for incremental rendering and can be memoized for performance. It also supports progressive updates and virtualized rendering if needed.

Accessibility: TanStack enforces accessibility props: you **must** provide an `ariaLabel` [【13†L268-L277】](#). It implements native keyboard navigation and ARIA roles for chart "points". This is a significant advantage: compared to other libraries, it aims to be WCAG-compliant by default. For example, each series can have a label, and focus can move between data points.

Testing: As the chart definition is a pure object, you can unit-test your transform logic and expected outputs easily. You can snapshot the SVG or HTML output (it uses a `<div>` host with SVG inside). Since it separates chart **definition** from **rendering**, you can also swap in mock renderers in unit tests. Visual tests (Chromatic) are recommended for final verification.

Fallback Libraries

- **react-chartjs-2 (Chart.js)** – A Canvas-based library with strong TS types. Use for dashboards with **real-time streams or >10K points** [【21†L329-L334】](#). Example snippet for a bar chart:

```

import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
  Tooltip,
  Legend,
} from 'chart.js';
import { Bar } from 'react-chartjs-2';

```

```

ChartJS.register(CategoryScale, LinearScale, BarElement, Tooltip, Legend);

const data = {
  labels: ['Q1', 'Q2', 'Q3'],
  datasets: [
    { label: 'Hires', data: [5, 8, 3], backgroundColor: '#2563EB' },
    { label: 'Leaves', data: [2, 4, 6], backgroundColor: '#D97706' },
  ]
};

export function StaffBarChart() {
  return (
    <Bar data={data} options={{ responsive: true }} />
  );
}

```

The Chart.js config API offers many plugins (zoom, decimation for downsampling). They recommend only loading the scales, elements, and plugins you need to minimize bundle [21†L329-L334] . By default, server-side rendering is not supported (Canvas requires a browser canvas).

- **visx (Airbnb)** – A collection of low-level SVG primitives (scales, axes, shapes). Use when you need *full* custom visuals. You import only the packages you need (e.g. `@visx/shape`, `@visx/scale`) to keep bundle small [24†L743-L748] . No chart component out-of-box means more dev work, but great for integrating precisely with your React state and Tailwind classes [24†L602-L610] .
- **Victory** – Another React/SVG library (MIT) with good documentation. Supports React Native parity. Use if you also target mobile apps. It has a decent SSR story (SVG) and a stable API. Compared to Recharts, Victory bundles are similar to moderate size [24†L749-L757] . No special React v4 is coming, but it's still maintained.
- **Nivo** – React components built on D3 (SVG/Canvas hybrid). Very polished and offers additional charts (geo, tree, choropleth). You can import only needed sub-packages (`@nivo/bar`, etc.) to save size [22†L425-L434] . It's popular in dashboards. SSR requires client-side (many charts rely on browser features) [22†L444-L452] .
- **AG Charts (Community)** – A high-performance chart library (MIT for community edition) that advertises full keyboard/screen-reader accessibility [20†L159-L164] . It supports downloads (PNG export) and many advanced types in the enterprise edition. Consider it if you already use AG Grid and need integrated charts.
- **Ant Design Charts** – If your HRMS uses Ant Design, this library (Canvas-based) will match its style. It has many built-in chart types but is relatively heavy [22†L518-L526] , so use only for enterprise internal dashboards where SSR is not needed.

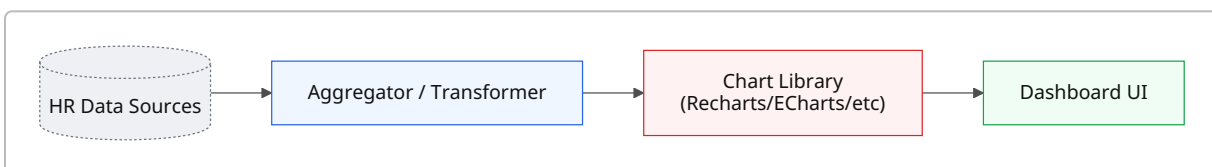
Theming & Design-System Integration

- All recommended libs let you style charts to match the neutral+blue HRMS palette. **Recharts/Visx/Victory** use SVG, so you can apply Tailwind classes (`stroke-[color]` , `text-[color]`) or CSS variables for fills and strokes. For example, use Tailwind's `stroke-primary` if you define `--tw-prose` colors, or inline style `stroke="#2563EB"` . Axes and grid lines can be styled via props or CSS (e.g. Recharts' `<CartesianGrid stroke="#E5E7EB" />`).
- **ECharts** and **Chart.js** use Canvas, so styling is via the configuration API (set `color` , `backgroundColor` , `fontFamily` , etc. in the chart options). Define a theme or set defaults: e.g. `ChartJS.defaults.font.family = 'Inter'; ChartJS.defaults.color = '#111827';` .
- **Dark Mode**: Each lib supports a dark theme: for SVG libs, switch className (e.g. add `dark` Tailwind class to parent). For ECharts, register a dark theme JSON. For Chart.js, set `ChartJS.defaults.color = '#A1A1AA'` etc.
- **Tailwind/shadcn**: Since Shadcn's charts are built on Recharts 【16†L169-L177】 , you can largely follow their example. Use the provided components (`<ChartContainer>` , `<ChartTooltipContent>`) or style your own. Ensure spacing and typography match (e.g. titles in `Inter` , `24px` , etc.) to keep consistency with the UI theme.

Performance & Large Data

- **Canvas vs SVG**: Use SVG (Recharts/TanStack/Visx) when dataset sizes are moderate (<~5k points) and SEO/SSR is needed. Use Canvas (Chart.js/ECharts/Ant) when you have very large series or frequent real-time updates 【21†L329-L334】 【24†L771-L775】 .
- **Virtualization**: For tables with embedded sparklines, only render sparklines for visible rows (e.g. with virtual scroll). Recharts itself doesn't virtualize, but you can use libraries like `react-window` to only mount chart components in view.
- **Downsampling**: Pre-aggregate or sample data on the client (or server) when plotting large time series. Some libs (Chart.js 3+) have decimation plugins you can enable.
- **WebGL**: If truly massive (100k+ points), consider a WebGL charting layer (outside scope of this list). Most JS libs don't use WebGL by default. ECharts can use its Canvas for ~thousands of points smoothly.
- **SSR**: For Next.js or similar, prefer libraries that support server-rendered output. Recharts, Victory, visx (pure SVG) and TanStack (SSR-friendly) are good. Chart.js/ECharts should only be used in client components, since their Canvas requires browser APIs 【21†L329-L334】 .

Mermaid Diagram – Data Flow:



This depicts how KPI data (e.g. payroll totals, headcount, attrition metrics) flows through a data aggregator (summing, grouping, time-windowing) into the charting component, which renders inside the React UI. For streaming updates, the aggregator would push new data to the chart periodically (e.g. via state or observables).

Accessibility Best Practices

- **ARIA Labels:** Always add `aria-label` to charts (especially Canvas ones) or wrap them with descriptive headings. For example: `<div role="img" aria-label="Monthly hires vs departures.">`.
- **Keyboard:** Use chart libraries' keyboard modes if available (e.g. `accessibilityLayer` in Recharts `【21†L261-L264】`). Verify that users can navigate between data points and rows in tables.
- **Color Contrast:** Ensure all chart colors (lines, bars, text) meet at least 4.5:1 contrast with the background (white or dark) `【17†L127-L136】`. Avoid encoding data in color alone; use patterns or labels in tooltips.
- **Focus Indicators:** If charts are interactive, show focus rings. For example, if a data point can be selected, style its outline (`strokeWidth` or `fill`) on `:focus`.
- **Captions:** Provide a textual summary of each chart (e.g. "Headcount has grown steadily from Jan to Dec"). This helps users with screen readers and cognitive disabilities.

Testing and Migration

- **Unit Tests:** Abstract chart configuration generation into functions and test them with Jest or similar. For React, render chart components with test data and assert that SVG elements (or Canvas calls) appear as expected. Use `data-testid` on wrapper divs if needed.
- **Visual Regression:** Use Storybook with a Chromatic or Percy service to capture chart snapshots for regression. Charts (especially analytical ones) benefit from pixel-level diff checks of critical views.
- **Integration Tests:** End-to-end tests (Cypress) can confirm that interacting with filters or timepickers updates the chart correctly (e.g. bars or lines match expected values).
- **Migration:** If replacing an existing chart solution, ensure mapping old data props to new APIs. For example, if migrating from Chart.js to Recharts, rewrite data structures (Recharts uses an array of `{x, y}` objects or keyed objects). Use the comparison table above to match features. Encapsulate charts behind wrapper components in your codebase so you can swap libraries by changing only that wrapper's implementation.
- **Performance Testing:** Use browser performance devtools or Lighthouse to measure CPU and paint times when charts render with large datasets. Adjust decimation or sampling if needed to keep UI interactive.

By following these guidelines, the chosen chart libraries can be integrated into your React/TypeScript HRMS frontend in a cohesive, performant, and accessible way. The final design will feel like a polished enterprise product (aligned with the UI/UX brief) rather than an "AI flashy" demo – charts will use the HRMS palette, minimal decoration, clear labels, and will integrate seamlessly into dashboards and reports.
